Git Stash Cheatsheet for Everyday Developers

Git stash acts like a temporary clipboard for code changes when you need a clean workspace fast. It prevents messy 'work in progress' commits and saves your sanity during sudden context switches.

What you'll learn:

1. Saving Work with a Message
2. Shelving Untracked and New Files
3. Restoring Changes (Pop vs Apply)
4. Inspecting and Deleting Stashes

Aman Raj Singh
Cloud Operations Engineer @ iManage
linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

1. Saving Work with a Message

A plain git stash works, but it defaults to an unhelpful name based on your last commit. Adding a message makes it crystal clear what you were building when you look back later. Always use the push subcommand with the -m flag to label your changes.

```
$git stash push -m "WIP: user auth redirect logic"
```

Git Stash: Save Your Work Without Committing

Tuesday, October 13, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

2. Shelving Untracked and New Files

By default, Git only stashes tracked files that have been modified. If you created new files, a basic stash leaves them untouched in your directory. Adding the -u flag tells Git to include all untracked files so your branch is truly spotless.

```
$git stash push -u -m "WIP: added payment gateway files"
```

Git Stash: Save Your Work Without Committing

Tuesday, October 13, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

3. Restoring Changes (Pop vs Apply)

When you are ready to resume work, you have two choices. Use 'pop' to apply changes and immediately remove them from your stash list. Use 'apply' if you want to restore the code but keep the backup stash intact just in case something breaks.

```
# Option A: Restore and delete
git stash pop

# Option B: Restore and keep
git stash apply stash@{0}
```

Git Stash: Save Your Work Without Committing

Tuesday, October 13, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

4. Inspecting and Deleting Stashes

Before restoring an old stash, you can inspect the exact code differences without applying them. Once you confirm you no longer need a saved state, drop it individually to keep your stash history uncluttered.

```
$# Inspect the diff
 git stash show -p stash@{0}

 # Delete a specific stash
 git stash drop stash@{0}
```

Git Stash: Save Your Work Without Committing

Tuesday, October 13, 2026

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Always attach a clear message using 'git stash push -m' so you recognize the work.
+ Use 'git stash -u' whenever your changes involve brand new files.
+ Run 'git stash list' before popping to ensure you restore the right stash index.
+ Prefer 'git stash apply' over 'pop' if you are unsure whether merge conflicts will occur.
+ Clear old stashes regularly to avoid accidental restoration of outdated code.

Watch Out For

! Assuming git stash saves new files: Untracked files are ignored unless you pass the -u flag.
! Treating stash as permanent storage: Stashes are local only and will be lost if your repository is re-cloned.
! Popping with merge conflicts: If conflicts occur, Git keeps the stash in the list; do not pop twice.
! Forgetting stash indices shift: When you drop or pop stash@{0}, stash@{1} becomes stash@{0}.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh